

Système de stockage distribué basé sur une DHT

Projet Master Systèmes et Applications Répartis (SAR)

PSAR – Sorbonne Université

Auteurs: Cedric Genevieve, Safa Gunes, Somaya Boudalil
Étudiants en Master 2026 — Systèmes et Applications Répartis

Instructeur: Prof. Alessio Pagliari

Date: 16 février 2026

Table des matières

1	Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué	2
1.1	Étude des approches P2P structurée: Les DHTs	2
1.1.1	Introduction au problème du Lookup (ref. Section 7.1)	3
1.1.2	Comparaison des stratégies de recherche (ref. Section 7.1.1 - 7.1.5)	4
1.1.3	Fondamentaux des Distributed Hash Tables (ref. Section 7.2)	6
1.1.4	Mécanismes des DHTs (ref. Section 7.3)	8
1.1.5	Interfaces des DHTs (ref. Section 7.4)	9
1.1.6	Conclusions (ref. Section 7.5)	10
1.2	Étude de l'approche hiérarchisée : HBase	11
1.2.1	Partitionnement par plages de clés	11
1.2.2	Avantages et limites	11
1.2.3	Positionnement par rapport aux DHT	12
1.3	Justification du choix architectural	12
2	Méthode de répartition des données	12
2.1	Motivation et définition de la répartition des données	12
2.2	Les différentes méthodes de répartitions des données	13
2.2.1	hachage classique	13
2.2.2	Hachage cohérent (Consistent Hashing)	15
2.3	Conclusion sur les méthodes de répartitions des données	19
3	Chord	19
3.1	Design du système et philosophie	19
3.2	Consistent Hashing et l'Identifier Circle	20
3.3	Localisation de clés scalable : Finger Tables	20
3.4	Théorèmes	21
3.5	Algorithmes des nœuds	22
3.6	Algorithmes de la Finger Table	24
4	Pastry	25

1 Analyse Comparative des Paradigmes Architecturaux de Stockage Distribué

L'explosion exponentielle du volume de données généré par les applications modernes a rendu obsoletes les architectures de stockage centralisées traditionnelles. Face à la nécessité de gérer des pétaoctets de données avec une haute disponibilité, les systèmes distribués se sont imposés comme la solution standard. Cependant, la distribution des données sur un grand nombre de nœuds introduit une complexité majeure : comment garantir la localisation efficace de la donnée (*lookup*), la cohérence (*consistency*) et la tolérance aux pannes (*fault tolerance*) dans un environnement où les machines sont intrinsèquement peu fiables ?

Dans le cadre de ce projet Master, nous nous intéressons à deux architectures de référence qui répondent à ces problématiques selon des paradigmes radicalement différents. Cette introduction analyse et compare ces deux modèles architecturaux considérés pour notre système de stockage :

1. L'architecture P2P structurée (Les DHTs) Ce premier paradigme repose sur une décentralisation totale (architecture *symmetric*). Chaque nœud joue un rôle identique, et le système s'auto-organise pour former un *overlay* structuré.

Plutôt que de nous limiter à une solution unique, nous étudierons plusieurs algorithmes de DHT (tels que Chord, mais aussi d'autres approches comme CAN, Pastry ou Kademlia). Bien que tous ces protocoles visent à résoudre le problème du *lookup* de manière scalable (généralement en $O(\log N)$ ou mieux), ils diffèrent par leurs topologies logiques (anneau, hypercube, arbre) et leurs stratégies de maintenance face au *churn*. Cette diversité permet d'analyser les compromis entre la latence de routage, la taille des tables d'état et la résilience du réseau.

2. L'architecture hiérarchisée orientée colonnes (Modèle HBase) À l'opposé de l'approche P2P pure, ce second paradigme adopte une architecture *master-worker* plus structurée, inspirée du Big-table de Google. Ici, le système est conçu pour offrir une cohérence forte et un modèle de données riche (orienté colonnes, versionné). Bien que distribué, ce modèle conserve des éléments de centralisation (via le HMaster et ZooKeeper) pour simplifier la gestion des métadonnées et des transactions. HBase illustre comment sacrifier une part de l'autonomie des nœuds permet d'optimiser les performances d'écriture et de scan sur de très larges ensembles de données.

La suite de ce document synthétise d'abord les mécanismes de routage et de maintenance des DHTs, avant d'aborder l'architecture interne de HBase et ses mécanismes de persistance.

1.1 Étude des approches P2P structurée: Les DHTs

Cette section s'appuie sur l'intégralité des notes de cours référencées afin de fournir une vue d'ensemble exhaustive du sujet. Ces notes sont considérées comme particulièrement complètes dans la mesure où elles couvrent l'ensemble du spectre technologique : de l'historique des systèmes distribués à l'adressage de bas niveau et au "*content-based routing*", jusqu'aux interfaces applicatives de haut niveau. Ce document offre une feuille de route permettant de comprendre comment les systèmes massivement distribués maintiennent leur stabilité sous une "*heavy network load*".

Référence : DBLP:conf/p2p/WehrleGR05.

1.1.1 Introduction au problème du Lookup (ref. Section 7.1)

Le document débute en notant une transition des systèmes distribués vers des réseaux massifs comptant des millions de participants, tels que Skype et BitTorrent. Ces systèmes doivent gérer la *"scalability, flexibility, and instant deployment"* sans aucun contrôle centralisé.

Au cœur de ces réseaux réside le **lookup problem**. Il est intéressant de noter que le document réduit cette problématique à une question unique :

"Où stocker, et comment trouver un certain élément de donnée D dans un système distribué sans aucun contrôle ou coordination centralisés ?"

L'objectif est qu'un nœud puisse récupérer un élément de donnée D spécifique sans avoir de connaissance préalable de l'emplacement actuel de D.

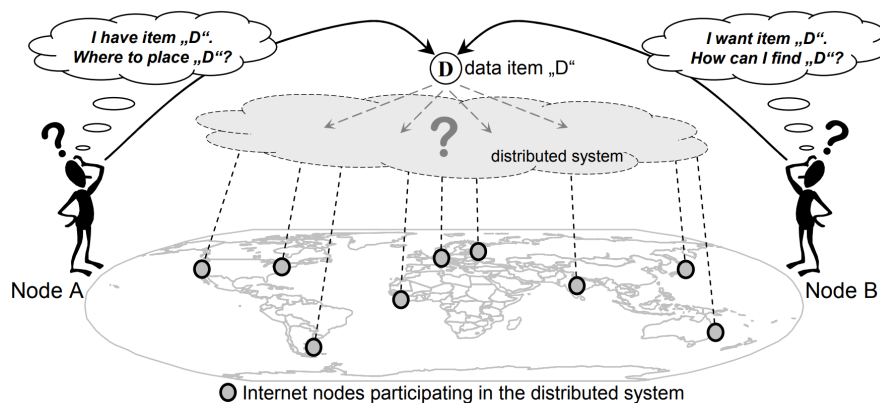


FIGURE 1 – **Le problème du lookup** : le nœud A souhaite stocker un élément de donnée D dans le système distribué. Le nœud B souhaite récupérer D sans connaissance préalable de l'emplacement actuel de D. Comment le système distribué — notamment le placement et la récupération des données — doit-il être organisé (en particulier au regard de la *scalability* et de l'efficacité) ?

1.1.2 Comparaison des stratégies de recherche (ref. Section 7.1.1 - 7.1.5)

Pour justifier l'usage des DHTs, le texte compare trois méthodes principales de gestion et de récupération des données :

1. Central Server

Cette approche de "première génération" (ex: Napster) implique un serveur central qui garde trace de l'emplacement de chaque élément.

En termes de performance, elle est très rapide, avec une complexité de recherche de $O(1)$. De plus, elle permet des *"fuzzy and complex queries"* car le serveur possède une vue globale.

Cependant, elle représente un *"single point of failure"*. Si le serveur échoue ou est attaqué, l'ensemble du système devient inutilisable.

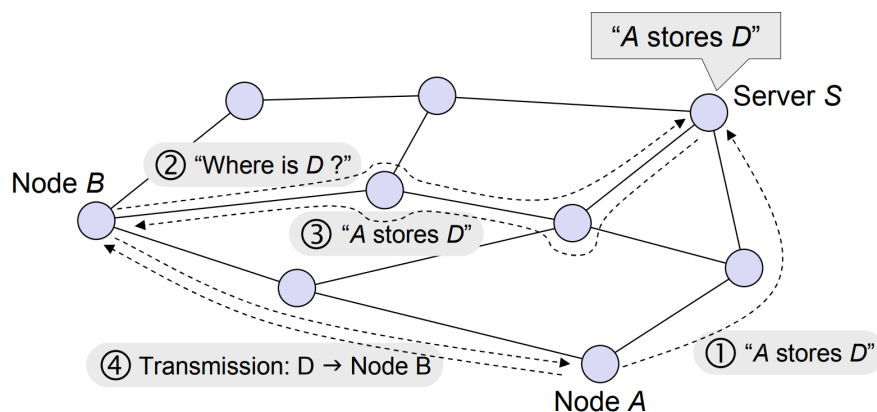


FIGURE 2 – **Central Server** : (1) Le nœud A publie son contenu sur le serveur central S. (2) Un nœud B interroge le serveur central S pour obtenir la localisation réelle d'un élément de donnée D. (3) Si celui-ci existe, S répond en indiquant la localisation de D. (4) Le nœud demandeur B télécharge le contenu directement depuis le nœud A.

2. Flooding Search

Utilisée par les systèmes de "deuxième génération" comme Gnutella, cette méthode supprime totalement le serveur.

Le mécanisme consiste en l'envoi par un nœud d'une requête à ses voisins, qui la propagent par *"flood"* aux nœuds suivants jusqu'à ce qu'un *"Time to Live"* (TTL) soit atteint.

Sa scalabilité est médiocre car l'*"communication overhead"* est extrêmement élevé. De plus, les résultats de recherche ne sont pas garantis car la recherche ne couvre qu'un nombre limité de *hops*, n'offrant donc pas la fiabilité requise pour ces applications.

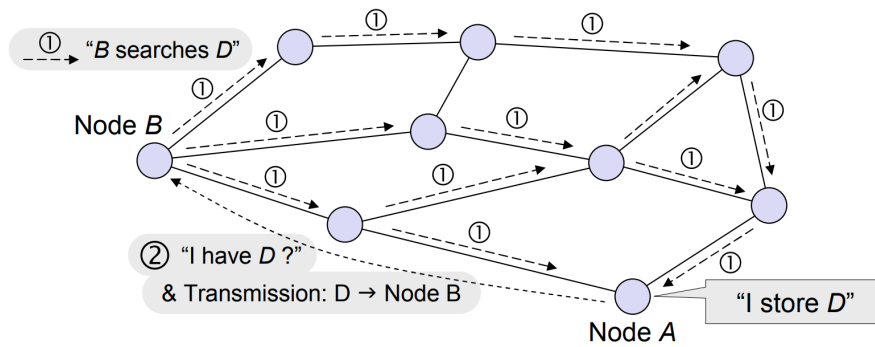


FIGURE 3 – **Flooding Search** : Aucune information de routage n'est maintenue dans les nœuds intermédiaires. (1) Le nœud A envoie une requête pour l'élément D à ses "neighbours" au sein du système distribué. Ces derniers transmettent la requête aux nœuds suivants de manière récursive (*flooding / breadth-first search*). (2) Le ou les nœuds stockant D envoient une réponse à A, puis A télécharge D directement depuis le ou les nœuds ayant répondu.

3. Distributed Hash Tables (DHTs)

Le document décrit les DHTs comme la "*golden path*" évitant les inconvénients des deux autres méthodes.

Elles sont souvent appelées "*structured Peer-to-Peer systems*" car elles utilisent des procédures proactives pour organiser le réseau. Chaque nœud ne gère qu'un petit nombre de références ($O(\log N)$), et les données peuvent être localisées en $O(\log N)$ hops.

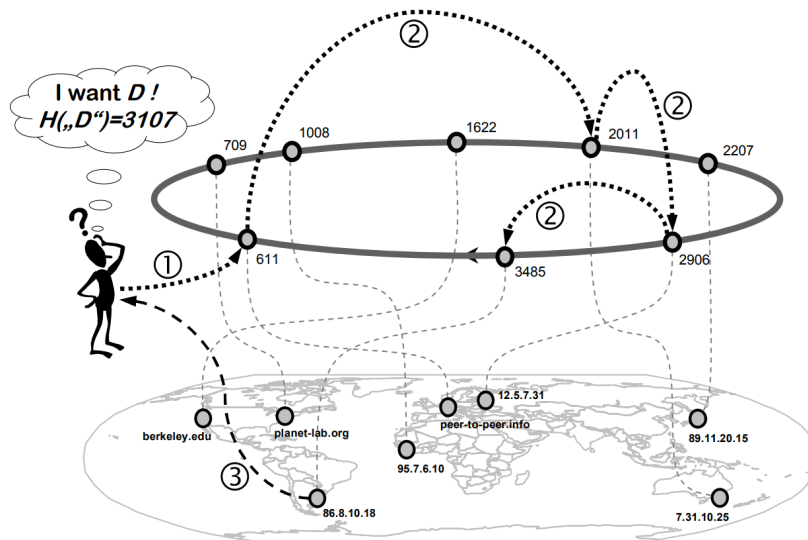


FIGURE 4 – **Distributed Hash Table** : Les nœuds du système distribué s'organisent en un *structured overlay* et établissent une quantité restreinte d'informations de routage afin d'assurer un routage rapide et efficace vers les autres nœuds de l'*overlay*. (1) Le nœud A envoie une requête pour l'élément D à un nœud arbitraire de la DHT. (2) La requête est transmise selon le routage de la DHT en $O(\log N)$ hops vers le nœud cible. (3) Le nœud cible envoie D au nœud A.

Réf : La Table 7.1 et la Figure 7.4 fournissent une comparaison directe de ces complexités, soulignant que les DHTs offrent le meilleur équilibre entre efficacité et robustesse.

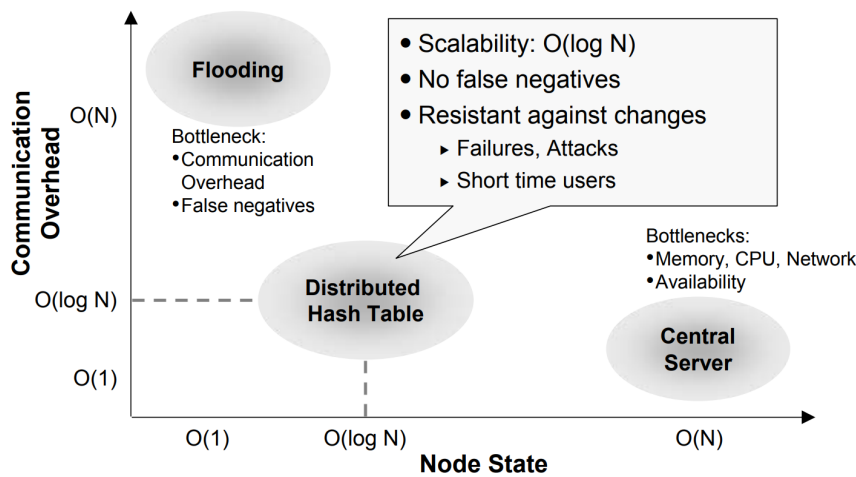


FIGURE 5 – Comparaison de la complexité en termes d'effort de recherche (axe y) et de coût de stockage par nœud (axe x). Les goulots d'étranglement (*bottlenecks*) ainsi que les caractéristiques spécifiques de chaque approche sont indiqués.

1.1.3 Fondamentaux des Distributed Hash Tables (ref. Section 7.2)

Cette section explore la stratégie proactive utilisée par les DHTs pour gérer les données, garantissant que chaque nœud possède un rôle spécifique et prévisible dans le réseau. Les DHTs gèrent les données en les distribuant entre les nœuds et en utilisant un "*deterministic routing scheme*".

Gestion distribuée des données (ref. Section 7.2.1)

Contrairement aux systèmes non structurés qui reposent sur un "*reactive flooding*" pour trouver des éléments, une DHT est "proactive".

Chaque nœud se voit assigner une "*range of data items*" spécifique dont il est responsable. Chaque nœud maintient une "*partial view*" de l'ensemble du système, stockant un nombre limité de liens vers d'autres nœuds.

Le processus de routage traverse plusieurs nœuds, se rapprochant de la destination à chaque *hop* jusqu'à ce que la cible soit atteinte.

Adressage dans les Distributed Hash Tables (ref. Section 7.2.2)

Les DHTs fonctionnent en créant un "*address space*" virtuel qui existe indépendamment de la topologie physique du réseau.

Ces espaces consistent généralement en de très grands entiers (par exemple de 0 à $2^{160} - 1$). Chaque nœud participant se voit attribuer une "*contiguous portion*" de cet espace d'adressage.

Les éléments de données reçoivent un "*identifiant ID*" unique dans le même espace, généralement en utilisant une "*collision-resistant hash function*" comme SHA-1 sur le nom du fichier ou son contenu.

Comme illustré dans la Figure 7.6, l'espace d'adressage est souvent interprété géométriquement comme un cercle ou un anneau.

Routage (ref. Section 7.2.3)

Le routage est la fonctionnalité centrale qui résout le *lookup problem*. Les nœuds ne connaissent pas tous les autres nœuds ; ils ne stockent qu'un "*bounded number of links*".

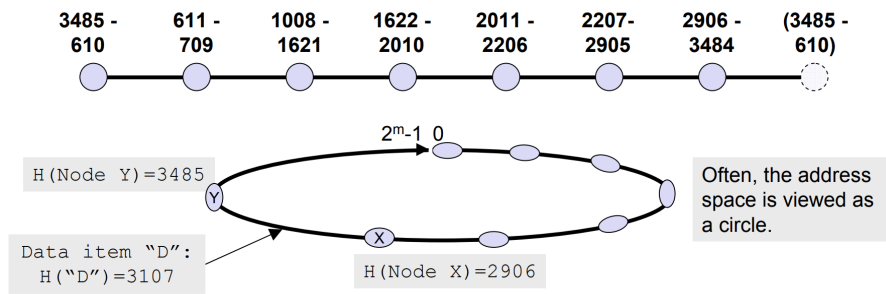


FIGURE 6 – Un espace d'adressage linéaire avec des valeurs entières allant de 0 à 65,535. L'espace d'adressage est partitionné entre huit paires.

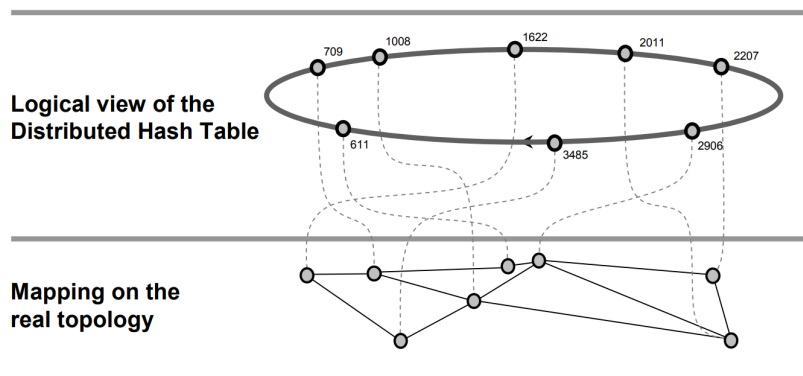


FIGURE 7 – Vue d'ensemble et vue du réseau sous-jacent (*underlay*) d'une Distributed Hash Table.

Si un nœud reçoit un message pour un ID qu'il ne possède pas, il le transmet au nœud de sa table qui est numériquement le plus proche de l'ID cible. Ce processus récursif est conçu pour trouver efficacement la destination en un "*small number of hops*".

*

Méthodes de stockage (ref. Section 7.2.4)

Il existe deux manières principales de stocker les données une fois le nœud responsable localisé, comme illustré dans la Figure 7.8 :

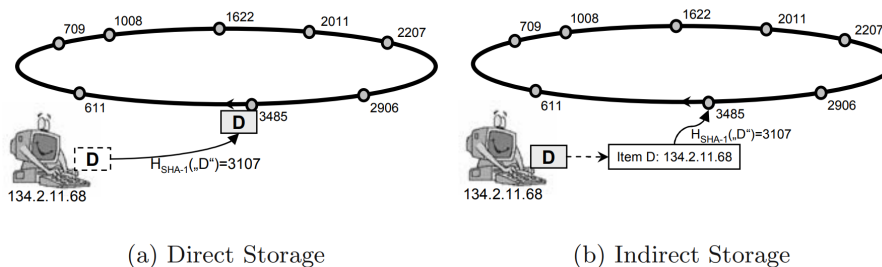


FIGURE 8 – Comparaison des deux méthodes de stockage des données au sein des Distributed Hash Tables : le *Direct Storage* et l'*Indirect Storage*.

Direct Storage : La donnée réelle est copiée sur le nœud responsable. C'est excellent pour la *availability* mais consomme plus de bande passante et de stockage.

Indirect Storage : Le nœud ne stocke qu'un "*pointer*" ou une référence vers l'endroit où la donnée est réellement conservée. Cela réduit la charge, mais signifie que la donnée est perdue si le nœud d'origine se déconnecte.

1.1.4 Mécanismes des DHTs (ref. Section 7.3)

Cette section se concentre sur la gestion des données et des nœuds, spécifiquement sur la manière dont le système gère l'arrivée, le départ et l'occurrence fréquente de *node failure*.

Avant qu'un nœud puisse interagir avec les données, il doit rejoindre le réseau. Cette arrivée déclenche des changements dans l'infrastructure, nécessitant la mise à jour des informations de routage et de la distribution des données. À l'inverse, le système doit détecter et s'adapter chaque fois qu'un nœud quitte le réseau ou échoue.

Arrivée de nœuds (ref. Section 7.3.2)

Le document spécifie un processus en quatre étapes pour qu'un nouveau nœud rejoigne la DHT :

1. **Bootstrap** : Le nouveau nœud utilise une méthode de *bootstrap* pour contacter un nœud existant arbitraire, qui sert d'*"entry point"*.
2. **Address Assignment** : Le nœud reçoit sa propre *"partition in the logical address space"*.
3. **Routing Update** : Les autres nœuds du système mettent à jour leurs informations de routage pour reconnaître le nouveau membre.
4. **Data Transfer** : Le nouveau nœud récupère les paires (*key, value*) dont il est désormais responsable auprès du nœud qui les détenait précédemment.

Défaillance de nœuds (ref. Section 7.3.3)

Parce que les systèmes distribués consistent souvent en des machines de bureau peu fiables et mal connectées, les défaillances sont fréquentes. Les DHTs gèrent cela via deux types de récupération :

Reactive Recovery : Si un nœud rencontre un pair défaillant lors d'une opération régulière, il gère la faute en trouvant des routes alternatives.

Proactive Recovery : Le système sonde périodiquement les autres nœuds (*"probes"*) pour vérifier leur statut. Si un nœud est non opérationnel, son entrée de routage est remplacée.

En cas d'échec d'un nœud, les données applicatives qu'il stockait sont perdues, à moins que la *"replication"* ne soit utilisée. Certains systèmes utilisent une approche *"soft-state"* où les données sont supprimées (*"pruned"*) à moins que l'application ne les rafraîchisse périodiquement.

Départ de nœuds (ref. Section 7.3.4)

Bien que les nœuds puissent être traités comme des *"failed nodes"* s'ils disparaissent, les DHTs préfèrent que les nœuds notifient le système avant de partir. Ce départ volontaire permet au système de copier immédiatement les données du nœud sortant vers son *successor*, de mettre à jour le routage plus rapidement (améliorant l'*"routing efficiency"*) et de garantir le bon fonctionnement des mécanismes de *"replication and load-balancing"*.

1.1.5 Interfaces des DHTs (ref. Section 7.4)

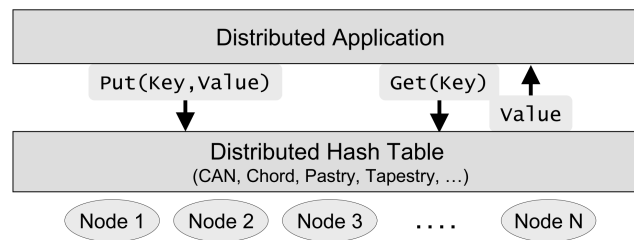


FIGURE 9 – Interface d’une Distributed Hash Table. À travers une simple interface put/get, la DHT fait abstraction de la distribution des données entre les nœuds.

Les notes de cours mentionnent deux angles principaux pour considérer la fonctionnalité d’une DHT : comme un système de routage ou comme un système de stockage.

Interface de routage (ref. Section 7.4.1)

Cette “*stateless interface*” se concentre sur la livraison de paquets basés sur un ID de destination plutôt que sur une adresse IP physique.

La primitive *send* accepte un ID de destination et un message, le livrant au nœud responsable depuis n’importe quel point du réseau. La primitive *receive* transmet les messages entrants et leurs identifiants à l’application sur le nœud récepteur. Cette interface étant générique, les développeurs peuvent implémenter leurs propres stratégies de stockage et de *load-balancing*.

Interface de stockage (ref. Section 7.4.2)

Cette couche ajoute de la complexité en gérant la persistance et la fiabilité des données. Comme le montre la Figure 7.9, elle imite une table de *hash* standard.

put prend une paire (*key*, *value*) et la stocke sur le nœud responsable. *get* accepte un identifiant et renvoie la valeur associée. La couche de stockage doit gérer les “*routing failures*”, prévenir la perte de données via la “*replication*” et gérer l’“*admission control*”.

Interface Client (ref. Section 7.4.3)

Le document note également que des nœuds peuvent agir comme de simples “*clients*”. Cela permet à des hôtes non-participants d’utiliser les services de la DHT sans rejoindre le réseau *overlay*, ce qui est utile pour la fiabilité ou l’“*access control*”.

1.1.6 Conclusions (ref. Section 7.5)

Le texte conclut en pesant les avantages significatifs des DHTs face à leurs défis persistants. Bien qu'elles résolvent les problèmes de *scalability* des systèmes centralisés, elles ne constituent pas une solution universelle.

Défis de conception

Les développeurs doivent naviguer entre plusieurs objectifs conflictuels :

- **Routing Efficiency** : Influencée par le nombre de références aux autres nœuds et la connaissance de la topologie au niveau IP.
- **Management Overhead** : Le coût de fond pour maintenir la DHT même en l'absence de trafic actif.
- **Churn** : Les taux élevés de *joins* et *leaves* simultanés exercent un stress particulier sur la stabilité globale, pouvant mener à des systèmes défectueux.

Limitations fondamentales

Malgré leur efficacité, les DHTs font face à des défis liés au principe de l'indexation distribuée :

- **Untrusted Environments** : Difficulté d'opérer de manière fiable lorsque les nœuds sont malveillants (*Byzantine faults*).
- **Query Constraints** : La récupération étant basée sur des identifiants numériques, effectuer des "*fuzzy searches*" ou des requêtes textuelles complexes est extrêmement difficile à réaliser efficacement.

1.2 Étude de l'approche hiérarchisée : HBase

Si les DHTs comme Chord offrent une réponse élégante au problème de la localisation dans un réseau instable, les besoins applicatifs nécessitent souvent des garanties plus fortes en termes de modèle de données et de persistance. C'est ici qu'intervient HBase.

Apache HBase est une base de données NoSQL distribuée et hautement scalable, construite au-dessus du système de fichiers distribué HDFS (*Hadoop Distributed File System*). Elle est conçue pour stocker et gérer de très grands volumes de données structurées ou semi-structurées tout en fournissant un accès rapide et aléatoire en lecture/écriture, ce qui lui permet de répondre aux besoins de systèmes analytiques ou transactionnels massifs. Son objectif principal est de permettre une scalabilité horizontale tout en conservant une cohérence forte au niveau des lignes.

HBase s'inscrit dans la famille des systèmes distribués à stockage partitionné, mais son approche diffère fondamentalement des architectures entièrement décentralisées comme les DHT. Là où les DHT privilégient un routage pair-à-pair basé sur le hachage cohérent, HBase adopte un partitionnement déterministe basé sur l'ordre des clés selon une configuration maître-esclave, ce qui influence fortement ses propriétés de performance.

1.2.1 Partitionnement par plages de clés

L'une des caractéristiques principales de HBase est le partitionnement horizontal des données : une table est découpée en régions, chacune couvrant une plage continue de clés (*row-key*). Chaque région contient un sous-ensemble des lignes de la table, ordonnées par clé, ce qui permet de répartir les données efficacement entre plusieurs nœuds. À mesure que les données augmentent, une région peut automatiquement se scinder en plusieurs régions plus petites afin de faciliter la distribution de la charge et d'améliorer les performances. Toutefois, la distribution dépend fortement du choix des clés. Si celles-ci suivent une progression monotone (par exemple des timestamps croissants), un phénomène de *hotspot* peut apparaître : toutes les nouvelles écritures sont dirigées vers la même région, ce qui crée un déséquilibre de charge temporaire.

Ce type de partitionnement par plages de clés (*range partitioning*) est différent du hachage cohérent (*consistent hashing*) utilisé dans les DHT (*Distributed Hash Tables*). Alors que la DHT répartit les données de façon pseudo-aléatoire pour équilibrer le chargement sans point central, HBase utilise un partitionnement déterministe basé sur l'ordre des clés, ce qui favorise les opérations séquentielles et facilite certains types de requêtes (requêtes par intervalle).

1.2.2 Avantages et limites

Un des principaux avantages de HBase est qu'il garantit une cohérence forte. Concrètement, lorsqu'une donnée est écrite, toute lecture suivante renvoie la version la plus récente.

Le choix d'un partitionnement par plages de clés permet également de bonnes performances pour les requêtes qui parcourent des données ordonnées. Comme les lignes sont stockées selon l'ordre des clés, il est efficace d'effectuer des recherches sur un intervalle ou d'analyser des données chronologiques.

Malgré ces points forts, HBase présente certaines limites. Le partitionnement par plages de clés dépend fortement du choix des clés. Si celles-ci sont mal conçues, certaines parties du système peuvent être plus sollicitées que d'autres, ce qui crée un déséquilibre de charge. À l'inverse, dans une DHT, le hachage permet généralement une répartition plus uniforme des données sans configuration particulière.

1.2.3 Positionnement par rapport aux DHT

HBase et les tables de hachage distribuées (DHT) partagent une préoccupation commune : la distribution de données à grande échelle. Cependant, leurs approches architecturales diffèrent. Les DHT reposent sur un principe de décentralisation forte : chaque nœud participe au routage et les données sont réparties grâce à un mécanisme de hachage, ce qui permet un bon équilibrage automatique. HBase adopte une approche plus structurée. Les données sont réparties selon des plages de clés et leur gestion est coordonnée. Cette organisation permet de conserver l'ordre des données et de garantir une cohérence forte, mais elle introduit plus de contrôle centralisé.

Autrement dit, HBase réalise un partitionnement déterminé et contrôlé avec une coordination explicite, tandis que les DHT réalisent un routage distribué sans maître.

1.3 Justification du choix architectural

SOMAYA ECRIT ICI!

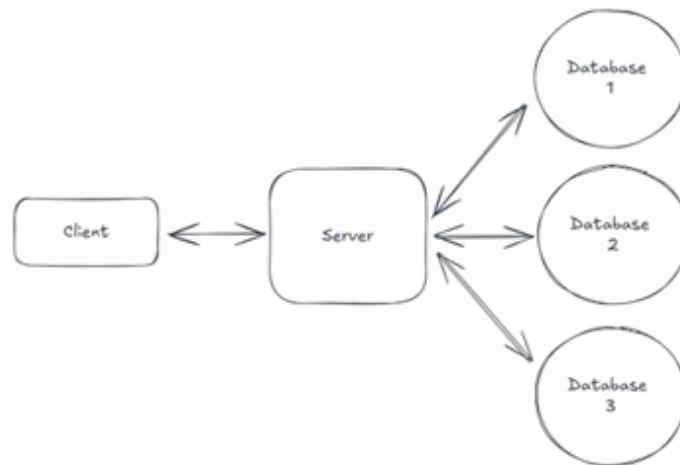
2 Méthode de répartition des données

2.1 Motivation et définition de la répartition des données

Avec l'augmentation continue du volume de données et du nombre d'utilisateurs, les architectures de stockage centralisées atteignent rapidement leurs limites. Une base de données unique n'a plus la capacité de contenir toute ces données et constitue un point de défaillance susceptible de compromettre la disponibilité du système.



Une solution pour pallier à ces limitations consiste à répartir les données et les requêtes sur plusieurs bases de données, afin de distribuer la charge de stockage et de traitement, et d'éviter la surcharge d'une base de données unique.



Toutefois, cette distribution soulève un problème fondamental : il devient nécessaire de déterminer de manière systématique sur quelle base de données une donnée doit être stockée et comment la retrouver efficacement. C'est pour répondre à cette problématique que les méthodes de répartition des données (data partitioning ou data sharding) ont été introduites. Elles définissent les règles de placement des données ce qui impacte directement les performances et l'équilibrage de charge. Ces méthodes de répartitions des données impactent également l'évolution du système dans le temps, c'est-à-dire lors de l'ajout ou le retrait d'une base de données.

2.2 Les différentes méthodes de répartitions des données

Après avoir montré la nécessité de répartir les données dans un système distribué, la première question qu'on se pose : comment associer chaque donnée à une base de données ?

Différentes méthodes de répartition des données permettent d'associer une clé à une base de données dans un système distribué.

Dans les exemples suivant nous allons utiliser le termes nœud qui fait référence à une instance base de données et le terme évènement qui fait référence à une donnée.

2.2.1 hachage classique

Principe général du hachage classique

Une première approche naïve appelée hachage classique (naive hashing ou modulo hashing) consiste à utiliser une fonction de hachage afin de faire la correspondance entre la clé d'une donnée et la base de données dans laquelle nous allons la stocker.

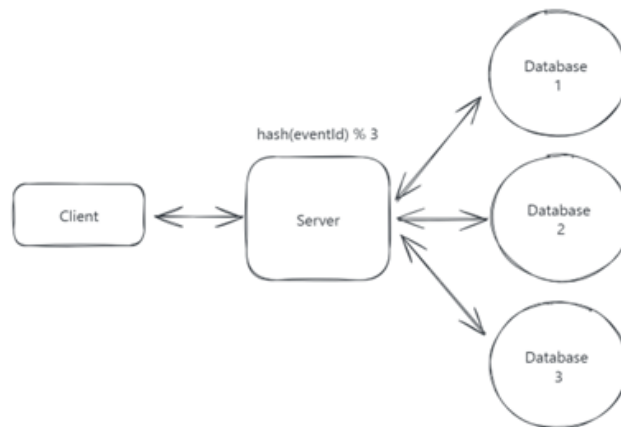
Association d'une clé à un nœud

Cette méthode repose sur les étapes suivantes :

1. Prendre l'id de l'événement et l'exécuter via une fonction de hachage, qui le convertit en nombre
2. Prendre ce nombre et effectuer l'opération modulo (%) avec le nombre de base de données.
3. Le résultat nous indique quelle base de données doit stocker cet événement.

La formule serait donc :

$$\text{database_id} = \text{hash}(\text{event_id}) \bmod \text{number_of_databases}$$



Pour un exemple concret avec 3 bases de données :

Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$

Event #5678 → $\text{hash}(5678) \% 3 = 0 \rightarrow \text{Database 1}$

Event #9012 → $\text{hash}(9012) \% 3 = 2 \rightarrow \text{Database 3}$

Ainsi, chaque clé est associée à une base de données unique, ce qui permet de retrouver l'emplacement d'une donnée à partir de sa clé.

Limite du hachage classique

Le premier problème survient lorsque l'on souhaite ajouter une nouvelle instance de base de données. La première solution à laquelle on pourrait penser consisterait à exécuter l'opération modulo avec 4 au lieu de 3 (Par rapport à l'exemple précédent).

$$\text{database_id} = \text{hash}(\text{event_id}) \% 4$$

La modification du nombre total de bases de données a non seulement affecté les données destinées à la nouvelle instance, mais a également entraîné une modification de la base de données associée à chaque événement précédemment stocké.

En effet, si on reprend un des exemples précédents :

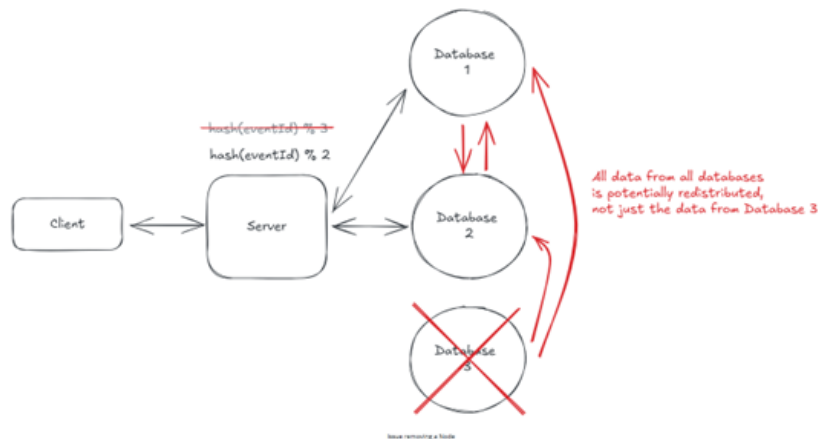
Event #1234 → $\text{hash}(1234) \% 3 = 1 \rightarrow \text{Database 2}$ $\text{hash}(1234) \% 4 = 0 \rightarrow \text{Database 1}$

L'événement n° 1234 était mappé à la base de données 2, mais avec l'ajout de la nouvelle base de données, ces données doivent donc être déplacées vers la base de données 1.



L'ajout d'une ou plusieurs bases de données entraîne donc une redistribution dans toutes les instances de base de données, ce qui provoque un mouvement massif de données inutiles. Ce mouvement massif cause d'énormes pics de charge de base de données et impact directement les utilisateurs. En effet, les utilisateurs ne pourront pas accéder à leur donnée ou auront un temps de réponse lent.

Un autre problème avec le hachage modulo simple réside dans le retrait d'une base de données. Supposons qu'une base de données soit tombée en panne. Il est nécessaire de redistribuer les données qui étaient dans la base de données tombée en panne vers les autres bases de données encore actives. Etant donné que le nombre de base de données actives diminue, le calcul pour trouver où ranger une donnée sera modifié. De ce fait, les données présentes dans les bases de données encore actives risquent d'être redistribuées vers d'autres instances de base de données.



Ces limitations montrent que le hachage modulo classique n'est pas adapté aux systèmes distribués dynamiques, où les bases de données peuvent être ajoutées ou retirées fréquemment.

2.2.2 Hachage cohérent (Consistent Hashing)

Principe général du consistent Hashing

Le consistent Hashing a été introduit pour résoudre le problème de la redistribution des données lors de l'ajout ou de la suppression d'une instance dans un système distribué. Le principe est d'organiser à la fois les données et les bases de données dans un espace circulaire appelé « anneau de hachage ».

Dans cet anneau, les clés et les nœuds sont positionnés en utilisant la même fonction de hachage. Chaque base de données se voit attribuer une position sur l'anneau, et chaque donnée est également positionnée sur cet anneau en fonction du résultat de sa fonction de hachage.

Association d'une clé à un nœud

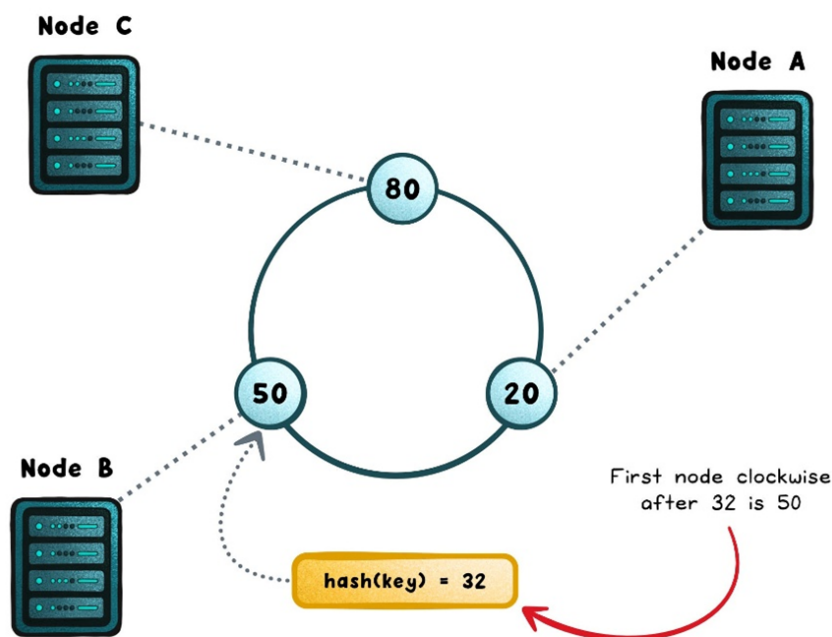
Cette méthode repose sur les étapes suivantes :

1. On réalise d'abord un anneau de hachage avec un nombre fixe de points.
2. On positionne les bases de données sur l'anneau en appliquant la fonction de hachage à leur identifiant.
3. Pour trouver sur quelle base de données une donnée doit être stockée, on hache d'abord l'ID de la donnée comme avec le hachage classique.
4. On se déplace ensuite dans le sens des aiguilles d'une montre sur l'anneau jusqu'à rencontrer le premier nœud.
5. Ce nœud va stocker de la donnée.

Par exemple :

Pour un nombre fixe de points égal à 90, 3 bases de données positionnées aux points 20, 50 et 80.

Si nous avons un $\text{hash}(\text{id_data}) = 32$, nous nous déplaçons dans le sens des aiguilles d'une montre pour trouver la première instance de base de données. Cette instance sera la base de données 2 (Node B) située au point 50.



Ajout d'une nouvelle base de données

Lorsqu'on ajoute une nouvelle base de données, seules les données qui se situent entre sa nouvelle position et la base de données précédente doivent être déplacées.

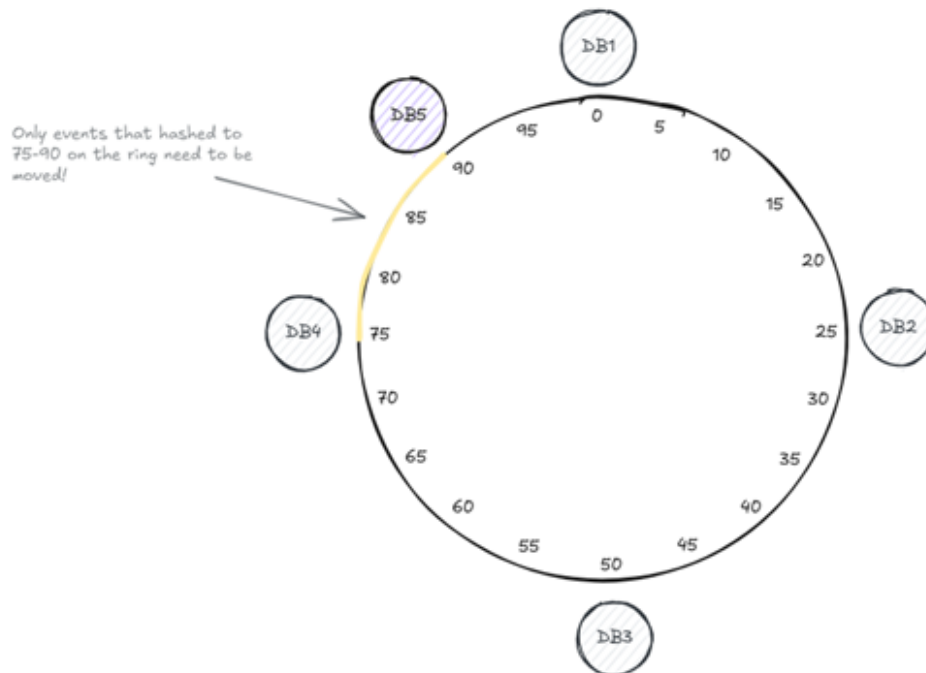
Par exemple :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Si nous ajoutons une cinquième base de données à la position 90 sur notre anneau :

- Seules les données qui se hachent vers des positions comprises entre 75 et 90 doivent désormais être stockées dans la base de données 5.

- Toutes les autres données restent dans leur base de données respective.



Hash Ring with DB5 added

Contrairement au hachage modulo classique, seule une petite portion des données est redistribuée.

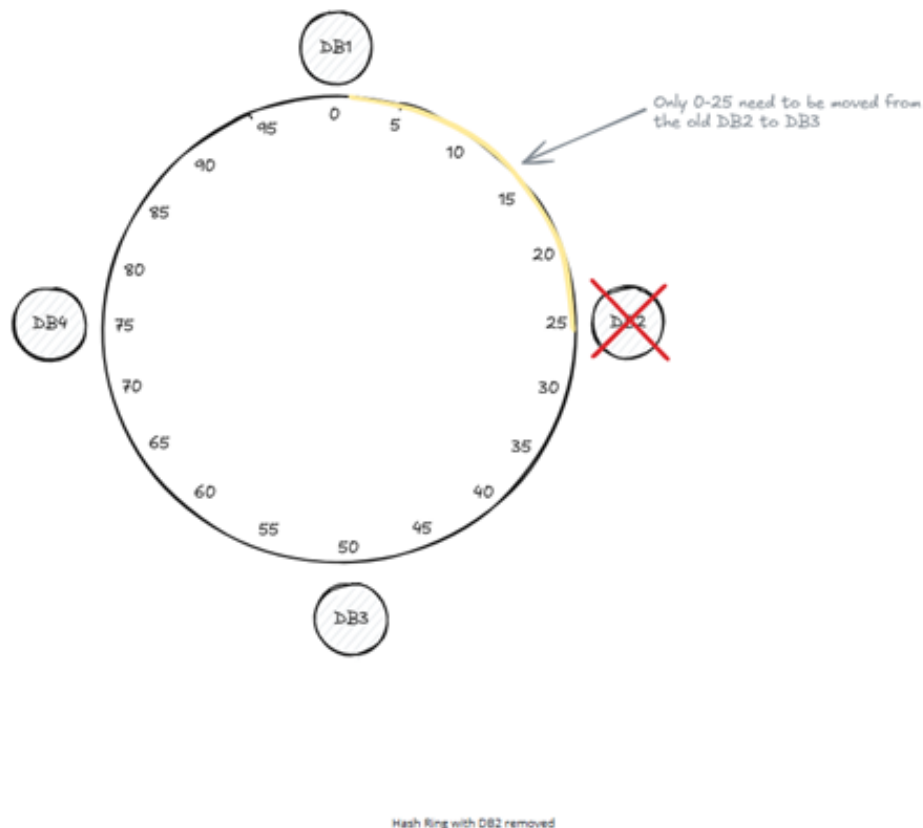
Suppression d'une base de données

En reprenant l'exemple initial :

Pour un nombre fixe de points égal à 100, 4 bases de données positionnées aux points 0, 25, 50 et 75.

Supposons que la base de données numéro 2 située à la position 25 rencontre une défaillance.

- Seules les données mappées sur la base de données 2 doivent être déplacées.
- Le déplacement se fait dans le sens des aiguilles d'une montre. Elles seront donc mappées à la base de données 3 située à la position 50.
- Toutes les autres données restent dans leur base de données respective.



Le consistent hashing permet ainsi de résoudre les principales limites du hachage classique en réduisant drastiquement le nombre de données redistribuées lors des changements de topologie (ajout ou retrait de base de données).

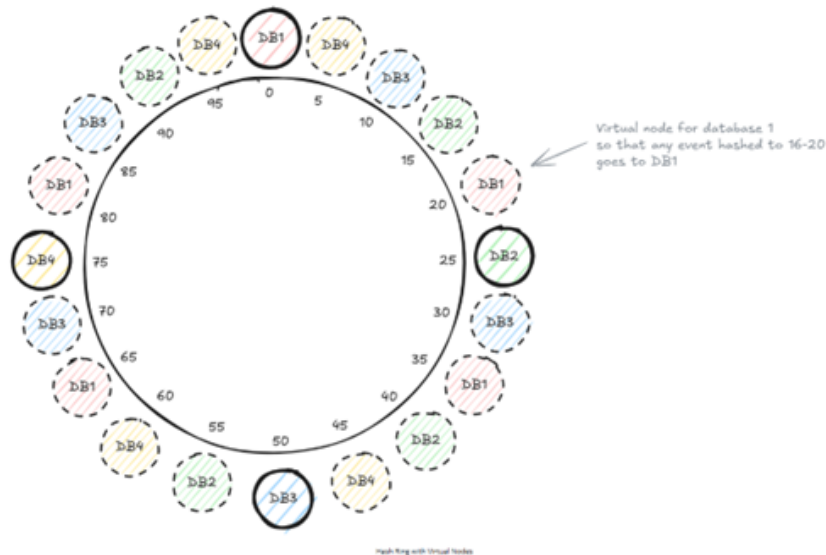
Cependant, il reste encore un problème. En effet, dans notre exemple ci-dessus où nous avons supprimé la base de données 2, nous avons dû déplacer toutes les données stockées dans la base de données 2 vers la base de données 3, ce qui peut entraîner une surcharge locale. Notre objectif serait de répartir de manière équitable les données lors de la suppression d'une base de données.

Optimisation : Nœuds Virtuels

Pour résoudre ce problème, une des solutions serait d'utiliser des « nœuds virtuels ». Au lieu de mettre chaque base de données en un seul point sur l'anneau, nous la divisons en plusieurs points en hachant différentes variantes du nom de la base de données.

Par exemple :

« DB1 » sera divisé en « DB1-vn1 », « DB1-vn2 », « DB1-vn3 » et « DB1-vn4 » qui seront respectivement aux positions 0, 20, 35, 65, 80. C'est le même principe pour les autres bases de données.



Nous pouvons observer sur la figure ci-dessus que les nœuds virtuelles sont mélangés autour de l'anneau. Supposons que la bases de données numéro 2 rencontre une défaillance :

- Les données mappées sur « DB2-vn1 » seront stockées dans la base de données 1
- Les données mappées sur « DB2-vn2 » seront stockées dans la base de données 3
- Les données mappées sur « DB2-vn3 » seront stockées dans la base de données 4
- Et ainsi de suite

La charge de la base de données supprimé est donc répartie de manière beaucoup plus homogène dans toute les bases de données actives.

2.3 Conclusion sur les méthodes de répartitions des données

Les méthodes de répartition des données sont donc indispensables dans les systèmes distribués. En effet, elles déterminent la manière dont les données sont réparties entre les bases de données. Le hachage classique est une solution simple, mais peu adaptée lors de l'ajout ou du retrait d'une ou plusieurs bases de données, en raison des redistributions massives qu'il engendre. Le consistant hashing permet de corriger ces limitations en réduisant les déplacements de données. Une des optimisations du consistant hashing repose sur l'utilisation de nœuds virtuels, permettant d'améliorer l'équilibrage de charge entre les nœuds lors du retrait d'une base de données.

3 Chord

Chord est un protocole de *lookup* distribué qui répond à un problème fondamental : localiser efficacement le nœud stockant une donnée particulière. Alors que de nombreux systèmes *peer-to-peer* offrent des fonctionnalités complexes comme l'anonymat ou le stockage redondant, Chord se concentre exclusivement sur une opération centrale : le *mapping* d'une *key* vers un nœud.

3.1 Design du système et philosophie

Chord est conçu comme une bibliothèque liée aux applications clientes et serveurs. Il répond à cinq défis critiques des systèmes *peer-to-peer* (P2P) :

1. **Load Balance** : Chord agit comme une fonction de *hash* distribuée, répartissant les *keys* uniformément pour éviter qu'un seul nœud ne soit submergé.

2. **Decentralisation** : Aucun nœud n'est plus important qu'un autre, ce qui élimine les *single points of failure* tels que ceux rencontrés dans Napster.
3. **Scalability** : Le coût du *lookup* croît selon le logarithme du nombre de nœuds, rendant possible l'exploitation de systèmes impliquant des millions de participants.
4. **Availability** : Le système s'ajuste automatiquement aux *joins* et aux *failures*, garantissant que le nœud responsable soit toujours localisé.
5. **Flexible Naming** : Chord utilise un *flat key space*, offrant aux applications une liberté totale quant à la manière de lier leurs noms internes aux *keys* de Chord.

3.2 Consistent Hashing et l'Identifiant Circle

Chord utilise une variante du *consistent hashing* pour assigner les *keys* aux nœuds.

Les nœuds ainsi que les *keys* reçoivent un **identifiant** de m bits via une fonction de *hash* de base telle que SHA-1. L'ID d'un nœud est dérivé de son adresse IP, tandis que l'ID d'une *key* est le *hash* de la donnée elle-même.

Les identifiants sont disposés sur un cercle modulo 2^m appelé **chord ring**. Une *key* k est assignée au premier nœud dont l'identifiant est égal ou supérieur à k dans le sens des aiguilles d'une montre. Ce nœud est appelé le *successor* de la *key* k .

3.3 Localisation de clés scalable : Finger Tables

Pour éviter l'inefficacité d'un parcours linéaire du cercle, Chord maintient une table de routage appelée **finger table**.

Chaque nœud n maintient jusqu'à m entrées. La $i^{\text{ème}}$ entrée pointe vers le premier nœud qui succède à n d'au moins 2^{i-1} .

Approche par Binary Search : Cette structure permet à un nœud de stocker davantage d'informations sur les nœuds qui le suivent de près et moins sur ceux qui sont plus éloignés.

Performance : Chaque *lookup* réduit de moitié la distance vers la cible, garantissant que la destination est atteinte en $O(\log N)$ *hops*.

Les nœuds proches d'une *key* disposent d'informations de type "*fine-grained*", tandis que les nœuds plus éloignés permettent des "*long jumps*" à travers le réseau pour accélérer la recherche.

3.4 Théorèmes

L'article présente 8 théorèmes servant à justifier l'usage du *consistent hashing* et de la structure de la **finger table**.

Théorème 1

Pour tout ensemble de N nœuds et K keys, avec une probabilité élevée : chaque nœud est responsable d'au plus $(1 + \epsilon)K/N$ keys. Lorsqu'un $(N + 1)^{\text{ème}}$ nœud rejoint ou quitte le réseau, seules $O(K/N)$ keys sont déplacées.

Ce premier théorème traite de l'efficacité avec laquelle Chord répartit les données sur le réseau, réalisant un "*natural load balance*". Il garantit qu'aucune machine ne devienne un goulot d'étranglement.

Théorème 2

Avec une probabilité élevée, le nombre de nœuds devant être contactés pour trouver un successor dans un réseau de N nœuds est en $O(\log N)$.

Ce théorème démontre l'efficacité des *lookups*, même dans des systèmes comptant des millions de nœuds. Il justifie la structure de la **finger table**, où chaque *hop* est conçu pour réduire de moitié la distance vers l'ID cible.

Théorème 3

Avec une probabilité élevée, tout nœud rejoignant ou quittant un réseau Chord de N nœuds utilisera $O(\log^2 N)$ messages pour rétablir les finger tables et les invariants de routage.

Le troisième théorème quantifie le coût du maintien de la structure du réseau lors de l'arrivée ou du départ de participants. Il permet aux développeurs d'estimer le "*management overhead*" du système. Il montre que bien qu'un *join* soit plus coûteux qu'un simple *lookup*, il reste très performant à mesure que le réseau croît.

Théorème 4

Dès qu'un nœud peut résoudre avec succès une requête donnée, il sera toujours en mesure de le faire à l'avenir.

Cette propriété garantit qu'une fois que le réseau a atteint un état où un chemin vers une *key* existe, les routines de maintenance du protocole ne briseront pas cette connectivité.

Théorème 5

Passé un certain délai après le dernier join, tous les successor pointers seront corrects.

Le protocole *stabilize* garantit la convergence, ramenant ainsi le réseau vers un état de cercle parfait une fois que le "*churn*" des nœuds entrants s'arrête.

Théorème 6

Si nous prenons un réseau stable de N nœuds, et qu'un autre ensemble allant jusqu'à N nœuds rejoint le réseau sans finger pointers (mais avec des successor pointers corrects), alors les lookups prendront toujours un temps en $O(\log N)$ avec une probabilité élevée.

Cela prouve que Chord est extrêmement résilient lors de phases de croissance. Même si un nombre massif de nouveaux nœuds rejoint le réseau sans avoir encore construit leurs **finger tables**, l'infrastructure existante garantit que la vitesse de recherche ne se dégrade pas.

Théorème 7

Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors avec une probabilité élevée `find_successor` renvoie le successor vivant le plus proche de la key demandée.

Ce théorème valide le mécanisme de **Successor List**. Il prouve qu'avec seulement une quantité logarithmique de redondance, le système peut localiser les données de manière fiable, même si la moitié des machines du réseau disparaissent simultanément.

Théorème 8

Si nous utilisons une successor list de longueur $r = O(\log N)$ dans un réseau initialement stable, et que chaque nœud échoue avec une probabilité de $1/2$, alors le temps attendu pour exécuter `find_successor` dans le réseau défaillant est en $O(\log N)$.

Il s'agit du pendant de performance du Théorème 7. Il garantit que non seulement la recherche trouvera le nœud correct après une défaillance de 50%, mais qu'elle le fera avec la même efficacité logarithmique.

3.5 Algorithmes des nœuds

1. L'algorithme de Successor Resolution

Le cœur de l'efficacité de Chord réside dans la fonction `find_successor`. Elle utilise la **finger table** pour effectuer des bonds exponentiels vers l'ID cible.

Algorithm: `Node.find_successor(id)`

```

1: if  $id \in (\text{node}, \text{successor}]$  then
2:   return successor
3: else
4:    $n' = \text{closest\_preceding\_node}(id)$ 
5:   return  $n'.\text{find\_successor}(id)$ 
6: end if
```

Pour trouver le nœud précédent le plus proche, le système itère à travers la **finger table**, du saut le plus grand vers le plus petit :

Algorithm: Node.closest_preceding_node(*id*)

```

1: for  $i = m$  downto 1 do
2:   if  $\text{finger}[i].\text{node} \in (\text{node}, id)$  then
3:     return  $\text{finger}[i].\text{node}$ 
4:   end if
5: end for
6: return node

```

2. Maintenance dynamique du ring : Joining et Stabilisation

Lorsqu'un nœud rejoint le réseau, il n'a besoin de connaître qu'un seul nœud existant pour s'intégrer au cercle. La fonction `stabilize` est exécutée périodiquement pour vérifier le *successor* et l'informer de l'existence du nœud.

Algorithm: Node.join(*n'*)

```

1: predecessor = nil
2: successor =  $n'.\text{find\_successor}(\text{node})$ 

```

Algorithm: Node.stabilize()

```

1:  $x = \text{successor.predecessor}$ 
2: if  $x \in (\text{node}, \text{successor})$  then
3:   successor =  $x$ 
4: end if
5: successor.notify(node)

```

3. Résilience : Le mécanisme de Successor List

Pour gérer le "*churn*" des nœuds tombant en panne sans préavis, chaque nœud maintient une liste de r *successors*. Cela garantit que si le *successor* primaire échoue, le nœud peut toujours router vers le prochain nœud actif disponible dans la liste.

Algorithm: Node.check_predecessor()

```

1: if (predecessor has failed) then
2:   predecessor = nil
3: end if

```

Algorithm: Node.handle_successor_failure()

```

1: for each  $s \in \text{successor\_list}$  do
2:   if  $s$  is alive then
3:     successor =  $s$ 
4:     break
5:   end if
6: end for

```

3.6 Algorithmes de la Finger Table

1. Calcul des entrées de la Finger Table

Chaque nœud n maintient une table de routage de m entrées. La $i^{\text{ème}}$ entrée est définie comme le premier nœud qui succède au point $n + 2^{i-1}$ sur le cercle d'identifiants.

$$\begin{aligned} \text{finger}[i].\text{start} &= (n + 2^{i-1}) \pmod{2^m}, \quad 1 \leq i \leq m \\ \text{finger}[i].\text{node} &= \text{find_successor}(\text{finger}[i].\text{start}) \end{aligned}$$

2. Initialisation de la Finger Table

Lorsqu'un nouveau nœud rejoint le réseau, il doit renseigner sa **finger table**. Alors qu'une approche naïve consisterait à appeler `find_successor` m fois, le protocole utilise souvent des nœuds existants pour fournir des "*hints*", réduisant ainsi la complexité de l'initialisation.

Algorithm: Node.init_finger_table(n')

```

1: finger[1].node = n'.find_successor(finger[1].start)
2: predecessor = successor.predecessor
3: successor.predecessor = node
4: for i = 1 to m - 1 do
5:   if finger[i + 1].start ∈ [node, finger[i].node] then
6:     finger[i + 1].node = finger[i].node
7:   else
8:     finger[i + 1].node = n'.find_successor(finger[i + 1].start)
9:   end if
10: end for

```

3. Maintenance proactive de la table

Le réseau étant dynamique, les entrées des doigts peuvent devenir "*stale*" lors de l'arrivée ou du départ de nœuds. La routine `fix_fingers` est exécutée périodiquement pour rafraîchir une entrée aléatoire, garantissant que les informations de routage demeurent *eventually consistent*.

Algorithm: Node.fix_fingers()

```

1: i = random index ∈ [1, m]
2: finger[i].node = find_successor(finger[i].start)

```

4. Mise à jour des autres nœuds

Lorsqu'un nouveau nœud n rejoint le réseau, il peut devenir le $i^{\text{ème}}$ doigt de plusieurs nœuds existants. La fonction `update_others` identifie ces nœuds et les force à mettre à jour leurs tables pour refléter l'arrivée de n .

Algorithm: Node.update_others()

```

1: for  $i = 1$  to  $m$  do
2:    $p = \text{find\_predecessor}(\text{node} - 2^{i-1})$ 
3:    $p.\text{update\_finger\_table}(\text{node}, i)$ 
4: end for

```

Algorithm: $p.\text{update_finger_table}(s, i)$

```

1: if  $s \in [\text{node}, \text{finger}[i].\text{node})$  then
2:    $\text{finger}[i].\text{node} = s$ 
3:    $p = \text{predecessor}$ 
4:    $p.\text{update\_finger\_table}(s, i)$ 
5: end if

```

4 Pastry